Sans savoir coder, on ouvre un fichier Word et on suit un cours pour prendre des notes… 2 chapitres intéressants ici : « Comprendre le web » et « Ligne de commande dans un terminal ». Le cours « HTML et CSS » n'est pas à comprendre ici sur ce word (retourner sur la page web pour le suivre).

Reconversion Web : ***Cours gratuit Openclassroom pour commencer*** (prof ChatGPT en bleu)

Comprendre le web

Internet : « Réseau » décentralisé, mails (ms sans l'interface)

différent web (= Cloud) : Service à « l'interieur » d'internet. www (lien hypertexte, la toile) Tim Berners-Lee (évolutions maintenant gérées par son W3C)

Autres services : Mails, Web, newsgroups (ancêtre des forum), FTP (échanger fichiers entre ordis)

Cloud (dématérialisation) : SaaS (logiciel utilisé à travers une interface web pour le grand public)

Langage :

Frontend (PC client): HTML (contenus), CSS (visuels), Javascript (interactions)

Backend (Serveur): PHP, Java, **Python**, Ruby …

Rm : Plus précisément y a des « niveaux » : Rôle :	*Monde **Python***	*Monde **JavaScript***
Langage :	*Python*	*JavaScript*
Exécution côté serveur :	*CPython…*	***Node.js***
Framework backend :	***Django**/FastAPI/Flask*	*Express/NestJS*
Gestionnaires de paquets :	*pip*	*npm*
Front-end :	*HTML/CSS/JS*	*HTML/CSS/JS,React,ect…*

Tels et tablettes : Site web « responsive » ou « Application mobile spécifique » (Si pas suffisant) .

Editeur de texte (=IDE) : Pour écrire son texte (**VSCode**…)

Bibliothèque (= librairie) : Morceaux de code écrits par d'autres développeurs pour gagner du tps.

Framework : Bibliothèque bcp plus élaborée (boite à outils pour backend). Ex : Python ->**Django**;

PHP -> Symphony, zend ; Ruby -> Rails; Java -> JavaEE (J2E) ; C# -> ASP .NET …

Base de données : Pour enregistrer informations utilisateurs (Grosse archive) : MySQL, PostgreSQL, Oracle, SQL Serveur… Faut un langage **SQL** . (encore derriere le « serveur »)

*Indépendant du langage ; Base de données : **PostgreSQL**, mySQL, MariaDB, SQLite, OracleDatabase…*

*Langage pr Base de données : **SQL**, NoSQL (MongoDB, Redis,Cassandra, Neo4j…),…*

Serveur = Data Center: Pour retrouver un serveur sur internet, on utilise un

Annuaire DNS : Lien entre « Adresse IP » (chaque ordi) et « Noms d'hôtes »(ex : google, **spterrain**) . Pour traduire le nom d'hôte en adresse IP.

Protocole : Pour que les ordis parlent entre eux.

De bas niveaux : TCP (le plus ancien), UDP

De hauts niveaux : **http**, https (s sécurisé), smtp (pr envoyer e-mail), ftp (échange fichiers)

Ex : http : 200 ok, 404 erreur

Métiers : Développeurs frontend, dev **backend**, QA manager, SysAdmin, Dev **Fullstack** (1 peu tout), dev **Low-code**, dev no-code, **vibe coder** (moi actuellement, un peu péjoratif), dev mobile, chefs de projets digitaux, product owners et product managers, UX designers , technicien informatique (SI : Système d'Information), professionnels DATA ou IA, marketing, communication… Les dev aiment utiliser ChatGPT.

Ligne de commande dans un terminal

Le code "durable" va dans le fichier en haut ; le terminal sert à exécuter et tester .

pwd : Afficher le répertoire de travail

ls : contenu d'un dossier

ls **-l** (pr plus d'infos) ; ls **-a** (même fichiers cachés) ; ls -la ; **cd** (changement de repertoire) **..** (dossier caché qui correspond au répertoire parent) ; cd nom_du_dossier

. : fichier caché **..** : fichier parent

Rm : Appuyer sur **Tab** (les 2 fleches qui s'opposent), permet de finir un mot.

Shift (c'est la fleche vers le haut pr les majuscules)

>>> veux dire que je suis en python. Taper **exit()** pr revenir à Git Bash

Mettre des **« »** si il y a un espace dans un nom. (ou **** avant)

Raccourcis : **ctrl+r** (Recherche ds l'historique des commandes) ; **ctrl + a** et **ctrl + e** (pr aller début ou fin commande)

***** : (wildcards, jokers) Remplace une partie du nom d'un fichier

Créer : **mkdir** : créer un nx répertoire

touch : créer un nx fichier (si ds un dossier : touch dossier/fichier.txt)

Déplacer : **mv** *elementADeplacer destination*

Copier : **cp** *elementACopier destination*

Supprimer : **rm** *elementASupprimer* Attention !!! , **-r** peut-être demandé en sécurité

Commande avancées : man (--help sur gitbash), cat, less, more (ancienne version de less)

Redirection : **>**

Grep : Chercher des éléments à l'intérieur d'un fichier.

Stop. Retourner sur la page web pour la suite.

Le reste de cette page est juste informatif et montre juste mes débuts débuts.

Créez votre site web avec HTML5 et CSS3

HTML (structurer le contenu .html) < > , CSS (mise en forme visuelle et dynamique .css) { }

Une ligne de code ds le html appellera le css.

Maîtrisez les bases de HTML5

Syntaxe HTML : < !DOCTYPE.html>

```
<html lang= « fr »>
    <head>
        <meta charset= « utf-8 »>
        <title> Le titre de ma page web <\title>
        <link href= « style.css » rel=  «stylesheet»>
    <head>
    <body>
        <h1 class=« gros-titre »> Bienvenue !<\h1>
        <p> Mon texte <\p>
    <\body>
    <\html>
```

< !-- Commentaire -->

1ere page web en html

Lancer un serveur ds le dossier : python -m http.server 8000

Puis ds chrome : http://127.0.0.1:8000/essaihtml.html

<p><\p>

< !DOCTYPE.html> ; **<html> <head>** <\head> **<body>** <\body><\html> Rm : <html lang="fr"> : permet être en francais

Tab (tabulation) = touche avec double flèches = Pour mettre début de la ligne de balise au bon endroit = indentation

Balises en paires : <title><\title> (title est pour l'onglet du navigateur -> sert pour résultats recherche google)

Balises orphelines : <img> ****

Attributs : options des balises : <img Nomdel'attribut = « Valeurdel'attribut »>

Balise ds balise : Parent (<html>) Enfant (<head>) car « dans » <html>.

<meta charset="utf-8"> : Indique l'encodage utilisé dans le fichier

Commenter son code : <!-- Ceci est un commentaire --> (« ctrl K » puis « ctrl C »)

Pour coder une balise → < > ; pour montrer < et > à l'utilisateur comme du texte → < et >.

« vrai titre » : h1

Liste : puis autant de qu'on a de points clés… Puis pr avoir une liste, on remplace juste ul par <ol> .

Mettez en forme vos pages web avec CSS3

Syntaxe CSS :

```
body {
    font-family : Arial ;
    color : black ;
    }
. gros titre{
font-size : 16px ;
    }
p : hover{
text-decoration : underline ;
    }
```

/* Commentaire */

Agencez le contenu de vos pages

Utilisez des fonctionnalités avancées de HTML et CSS

Je commence maintenant à coder… Je reprend HTML et CSS en code !!!

Rm : Comme SPTerrain est en « Python », j'ai même commencé par le chapitre suivant… Je me suis rapidement retrouvé bloqué.
J'ai donc repris depuis les bases ci-dessus et le chapitre « HTML et CSS » est maintenant en ligne.
Pour information, voilà comment j'ai commencé, mais Python sera maintenant abordé après avoir vu les bases 😉.

Apprenez les bases du langage Python

1-3) Créez des données

Installer environnement : https://code.visualstudio.com/ + Extensions : Python … python.org (cocher la case "Add to path" si on est sous Windows)

https://git-scm.com/.

Editeurs de texte : Sublime Text (ressource en anglais)

Afficher quelque chose à l'écran : **print**("hello, world!")

Créer (= déclarer = initialiser) une variable : **nom = « valeur »** (*ex* : livre= « Gatsby le magnifique »)

f-string : Pr afficher des variables dans du texte.

Exemple : print(f"Je m'appelle {prenom} et j'ai {age} ans.")

ou print(f"Le résultat est {a + b}")

Rm : Les virgules deviennent des points (anglais) : Ex : 12,4 en français, 12.4 ds Python.

Booléens : True ou False

type() : Permet de voir rapidement avec quelles données on travaille.

Regrouper plrs éléments ensembles : **Ma_liste [, ,]** Rm : Indice commence à 0 (pas à 1), index

Ex : plateformes_sociales = ["Facebook", "Instagram", "Snapchat", "Twitter"]

>>> plateformes_sociales[0] "Facebook"

>>> plateformes_sociales[-1] "Twitter"

Ma_liste**.append(**"Linkdn"**)** : Ajouter ; **.remove()** : Retirer ; .**len()** : Connaître longueur ; .**sort()** : Pr tri ;

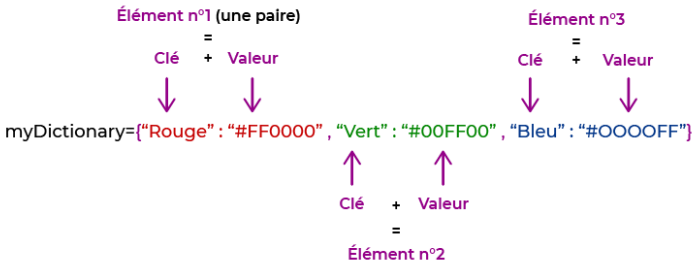
… **documentation Python**.

.extend()	Ajoute plusieurs éléments à la fin de la liste.
insert()	Insère un élément à une position donnée dans la liste.
pop()	Supprime et renvoie l'élément à une position donnée de la liste, ou le dernier élément si aucun indice n'est spécifié.
index()	Renvoie la première occurrence de l'élément spécifié dans la liste.
count()	Renvoie le nombre d'occurrences de l'élément spécifié dans la liste.
reverse()	Inverse l'ordre des éléments de la liste.

Les « tuples » : **()** au lieu des crochets [ ] en Python.

Rm : Ils sont immuables (alors que les « listes » sont modifiables)

Dictionnaires : paires clés-valeurs **{« » : « », }**



Ex (ou): 1 nouvelle_campagne = { 1 taux_de_conversion = {}

2 "responsable_de_campagne": "Jeanne d'Arc", 2 taux_de_conversion['facebook'] = 3.4

3"nom_de_campagne": "Campagne nous aimons les chiens",

4"date_de_début": "01/01/2020",

5"influenceurs_importants": ["@MonAmourDeChien", "@MeilleuresFriandisesPourChiens"]

6}

Manipulations du dictionnaire : nom_du_dictionnaire

.keys()	Retourne une vue sur les clés du dictionnaire.
.values()	Retourne une vue sur les valeurs du dictionnaire.
.items()	Retourne une vue sur les couples (clé, valeur) du dictionnaire.
.get(clé)	Retourne la valeur associée à la clé spécifiée. Si la clé n'est pas présente dans le dictionnaire, retourne la valeur None .
.pop(clé)	Supprime la clé spécifiée et retourne la valeur associée. Si la clé n'est pas présente dans le dictionnaire, retourne la valeur None .
.clear()	Supprime tous les éléments du dictionnaire.

in : pour vérifier si une clé spécifique existe dans un dictionnaire.

2-3) Gérez la logique du programme

Déroulement du programme avec conditions : l'ordre

If, if/else, if/elif/else

And, or, not

Égal à : a == b ; Non égal à : a != b ; Moins que : a < b ; Moins que ou égal à : a <= b ; Plus que : a > b ; Plus que ou égal à : a >= b (Attention, = assigne une valeur)

Les match cases (pr faciliter comparaisons de valeurs) : _

Boucles (pour répéter les taches) : for , *fonction* range () , while (jusqu'à une condition), break (sortir boucle prématurément), continue (passer à prochaine itération de la boucle)

Fonctions (bloc de code) : (argument)

Créer la fct : def message() :

Print(« Hello, World! »)

Ou print (« nom:» , nom)

Print(« prenom :»,prenom)

L'appeler : message ()

Ecrire un code propre : DRY (**D**on't **R**epeat **Y**ourself) , séparer les responsabilités, commenter son

Code (ce qu'un morceau de code est censé faire) # (commentaire à ligne unique avant et après)

Docstrings (description fonction) : « « «

Erreurs et exceptions : try/except

Importer Packages : librairie = ce que ça fait pour toi (ensemble de code réutilisable)

package = comment le code est emballé / organisé (paquet installable)

En pratique, ces 2 mots sont utilisés comme synonymes.

Import (pour l'importer), pas mettre * car ça importe tout et c'est dangereux.

Dépôts en ligne tels que **PyPI** (gestionnaire de packages **pip**) : pip install PyPI

Extraction web : ETL (extract transform load)

Prérequis : Créez votre première page en HTML (plus haut dans « Créez votre site web avec HTML5 et CSS3 »)

3-3) Extraire des données à partir du web avec les packages Python

Lexique commandes terminales

- *pwd* : permet d'afficher le répertoire courant ;
- *ls* : permet d'afficher le contenu d'un répertoire ;
    - *ls -a* : l'option -a affiche également les fichiers et dossiers cachés,
    - *ls -l* : l'option -l modifie l'affichage pour rajouter de nombreuses informations ;
- *cd dossier* : permet de se déplacer à l'intérieur d'un répertoire ;
- *mkdir dossier* : permet de créer un dossier ;
- *touch nomFichier* : permet de créer un fichier ;
- *mv source destination* : permet de déplacer des éléments ;
- * : est un caractère qui peut être utilisé comme substitut pour n'importe quel caractère dans une recherche ;
- *cp source destination* : permet de copier des éléments ;
    - *cp -r* : l'option -r permet de copier un répertoire ;
- *rm fichiers* : permet de supprimer des fichiers ;
    - *rm -r dossiers* : l'option -r permet de supprimer des répertoires ;
- *man commande* : permet d'afficher le manuel d'une commande ;
- *cat/less/more nomFichier* : permet d'afficher le contenu d'un fichier ;
- > : permet de rediriger le résultat d'une commande vers un fichier ;
- *grep motif chemin* : permet de faire des recherches dans des fichiers.